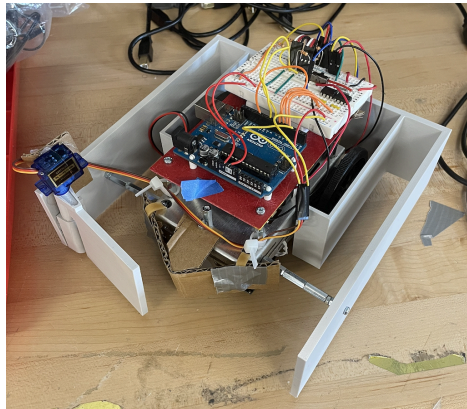


Team #46:

Charles Wall-Davis (caw362), Langston Johnson (lcj36), and Joshua Delaney (jpd298)

MAE 3780 Robot Project Final Report

1. Robot Design and Strategy Overview:



The main mechanical aspects of our robot are the wheels and the door. The doors were meant to open up at the beginning of the match and to remain open in order to collect blocks. The doors would close if the robot needed to back up (either it be due to an opposing robot or the QTI sensors detecting the border). We decided to elevate the arduino board above the robot and place our 6V battery pack along with our 9V battery under the slot to make wiring easier.

2. Design Process Reflection:

In the beginning stages of our design process, we created rough sketches of what we wanted our robot to look like. First and foremost, we wanted to design an exterior shell for our robot which would serve as both a shield and cube container (as seen in the sketches in appendix c). In addition, we wanted to create a door mechanism that we could attach to the exterior in order to aid in accumulating cubes and to prevent them from being lost. Since we did not necessarily need the exterior to complete the first 3 milestones, we did not go through a prototyping phase and decided to CAD (see appendix c), and eventually 3D print the final exterior design for milestone 4. We were able to attach the exterior to the chassis of the robot and we were able to print out the doors. Unfortunately, we broke one

of our doors while attempting to attach them to the exterior and we were only left with one during the competition.

3. Competition Analysis:

During the day of the competition our robot achieved a record of 2 wins and 5 losses during our 7 match round-robin. Our robot performed the best during the sixth match in which it started on yellow and acquired 6 blocks to our opponents 0. Our losses were due to either our robot starting on blue and misreading blue as black, resulting in running off the course, or getting stuck in a corner. Due to earlier complication with the code the morning of competition, our door's function to sweep blocks in was unsuccessful, however, our exterior design was able to successfully capture blocks within our perimeter. Our turning system for our robot didn't work very well as sometimes it would drive us off the board or get us stuck in a corner. Unexpectedly, our robot performed better on the yellow starting side and lost every time it was on the blue starting side. The strength of our design was the sturdy exterior and space inside for blocks to sit. Our weakness in our design was keeping the blocks inside, maneuverability, and lack of ultrasonic sensors.

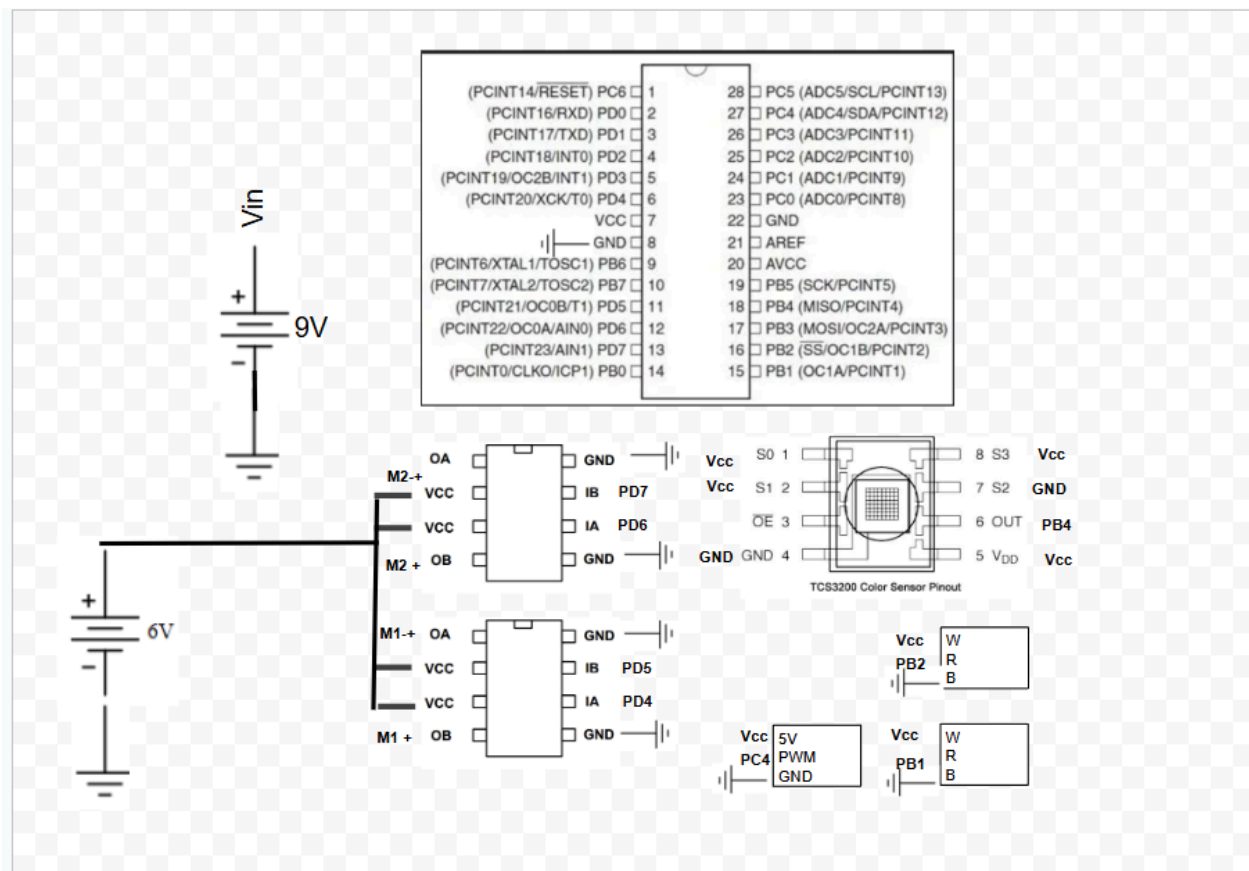
4. Conclusions:

In conclusion, our team had a good idea for our design in theory but our performance was hindered by our execution of the design. If we could go back and change our design, we would have the centered wheel towards the back so that we could collect blocks underneath our robot. The addition of taller wheels could help with that. Additionally, we would construct our exterior with the 12in diameter circle requirement in mind so that we don't run into the same conflict as before. Lastly, we would have two fully operating doors to adhere to our initial design intentions. As advice for future students doing this project, make sure to save your code and have multiple sources of back up so that you don't have any unsaved code issues on the day of competition. Additionally, make sure to read the rules on the restrictions of your robot so cuts don't have to be made day off. Lastly, if you want to ensure your under budget, cardboard is a fairly good material to use in contrast to acrylics and 3D printing.

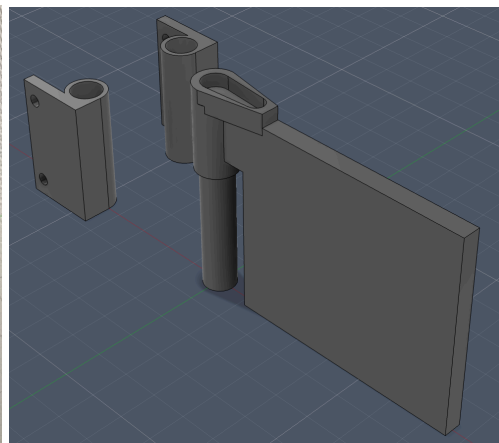
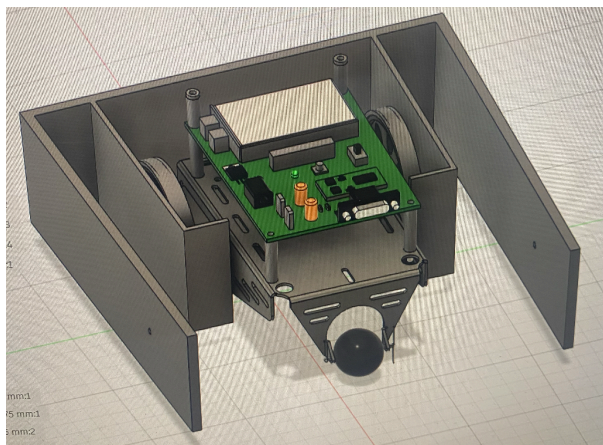
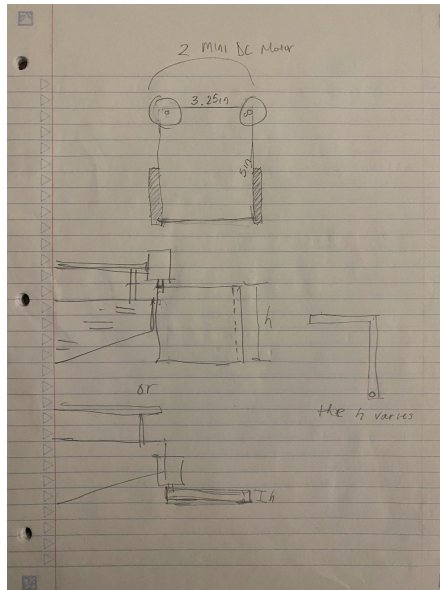
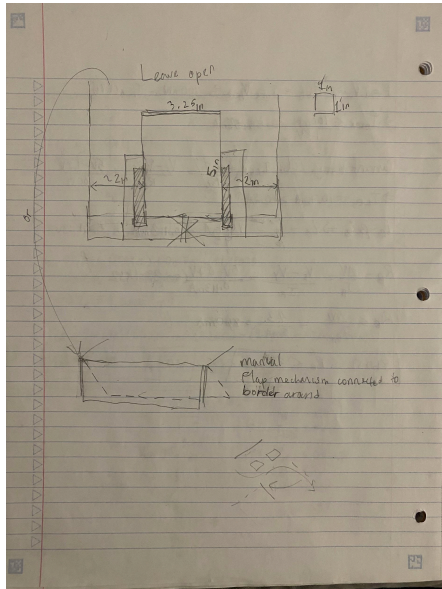
5. Appendix A

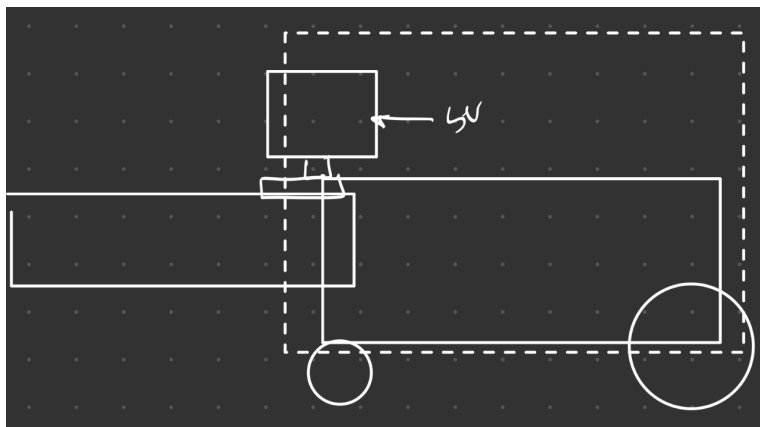
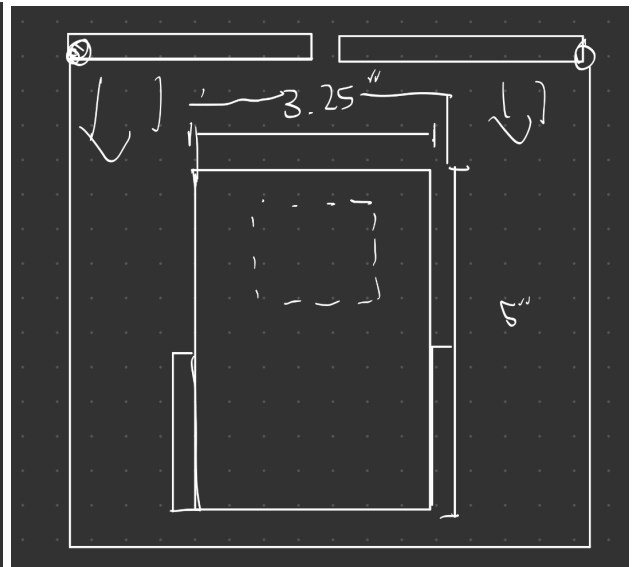
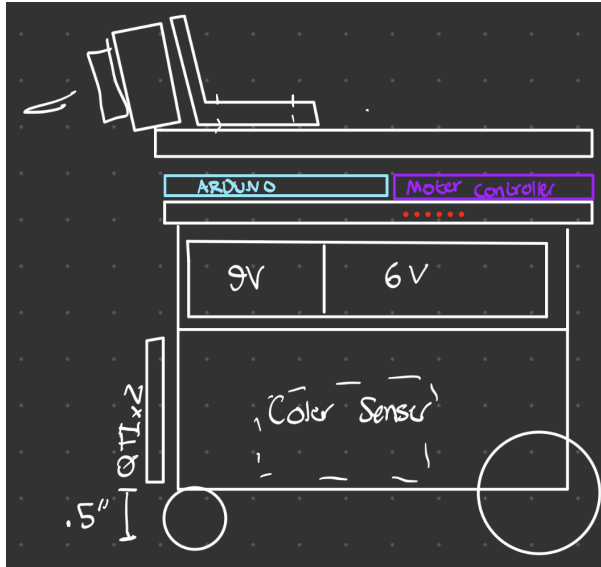
Part	From	Quantity	Dimensions	Cost
2-Pack Wheels (Part #: 4925)	Pololu	1	70×8mm	\$7.75
Robot Exterior	RPL	1	200x200x60mm	\$86.48
Door + Axle	RPL	2	82x12x59mm	\$10.52
Door Base	RPL	2	22x14x35mm	\$3.10
Positional Servo Motor	Lab	1	-	\$4.00
Total Cost				\$125.47

6. Appendix B:



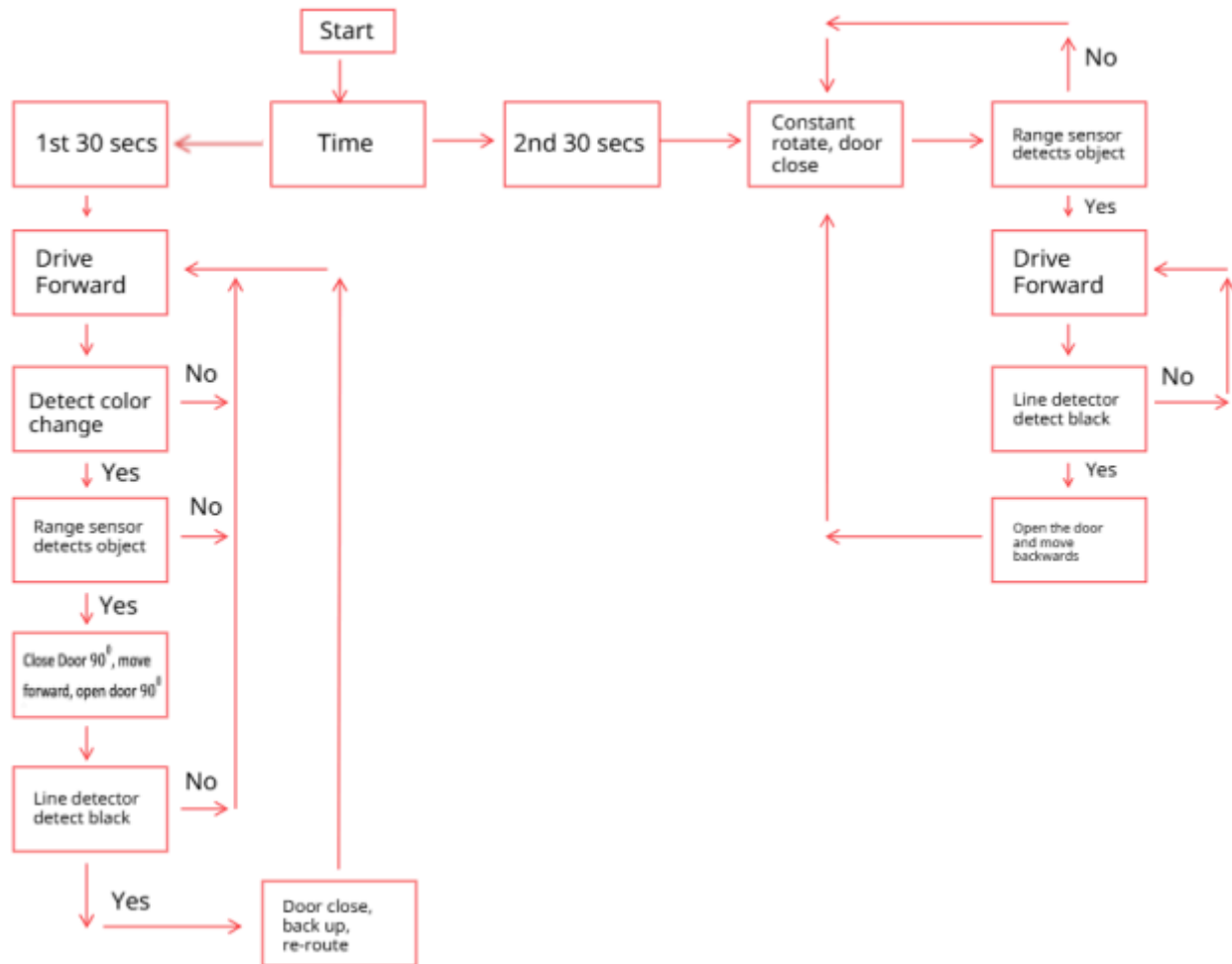
7. Appendix C:





Exterior	Door+Axle	Door Base
Mass 169.961 g Volume 2.158E+05 mm³ Physical Material ABS Plastic (1)	Mass 18.047 g Volume 22916.462 mm³ Physical Material ABS Plastic (1)	Mass 3.205 g Volume 4069.797 mm³ Physical Material ABS Plastic (1)
169.961(.5) + 1.5 = 86.48	18.047(.5) + 1.5 = 10.52	3.205(.5)+1.5 = 3.10

8. Appendix D:



9. Appendix E:

```

bool active = true;
int main(void) {
    Serial.begin(9600); //Starts serial communication over 9600 baud
    /*
    main -
        MONITORS - Measure and store sensor data
        CONTROL TASKS - Controls actuators
        SETUP - Code for monitors and control tasks that run once
        EXECUTE - Code for monitors and control sensors that run repeatedly
        GAME MANAGER - Executes the game strategy using data from sensors to
inform the operation of the control tasks
  
```

```

    */

    edge_sensor_setup();
    color_sensor_monitor_setup();
    drive_control_setup();
    door_control_setup();

    open(); //opens door

    while(active){
        color_sensor_monitor_execute();
        edge_sensor_execute();
        door_control_execute();
        game_manager_execute();
    }
}

// Edge and Color sensors both have functions that execute in response
// to INT0 vector

ISR(PCINT0_vect){
    edgeInterrupt();
    colorInterrupt();
}

void milestone1_execute(){
    /*
    1. Drive forward 1 foot (0.3048 m)
    2. Turn right 90 degrees
    3. Drive forward 1 foot (0.3048 m)
    4. Turn left 90 degrees
    5. Drive forward 6 inches (0.1524 m)
    6. Drive backward 1.5 feet (0.4572 m)
    7. Turn left 90 degrees
    8. Drive forward 1 foot, returning to the starting position
    9. Stop
    */
}

```

```

    drive_forward(0.3048);
    turn_right(90);
    drive_forward(0.3048);
    turn_left(90);
    drive_forward(0.3048/2);
    drive_backward(0.3048*1.5);
    turn_left(90);
    drive_forward(0.3048);
    stop();
}

void milestone4_execute() {
    /*
    Drive forward .32 meters. Turn right 80 degrees drive forward a foot
    */
    drive_forward(0.32);
    turn_right(80);
    drive_forward(0.3048);
}

void pause() { //Setting PORTD to 0b00000000 sets all the h-bridge input
pins to low, causing the both wheels to stop moving
    PORTD = 0;
}

void stop() { //Stops the robot from moving and ends the while loop. No
code will execute after this function is called
    pause();
    active = 0;
}

int game_state = 1; //Begin with the drive forward game state
void set_game_state(int state) { //setting function for the game state
    game_state = state;
}

int get_game_state() { //getting function for accessing the game state
    return game_state;
}

void game_manager_execute() {
    Serial.println(game_state);
    if(game_state==0) { //STOP

```



```

        drive_backward(0);
    }
    else if(game_state==1){ //DRIVE FORWARD
        drive_forward(.002);
    }
    else if (game_state==2){ //AVOID LEFT EDGE
        drive_backward(.025); // drive backwards 5 centimeters
        turn_right(20); //turn left 1 degree
        set_game_state(1); //transition to game state 1 - DRIVE FORWARD
    }
    else if (game_state==3){ //AVOID RIGHT EDGE
        drive_backward(.025); // drive backwards 5 centimeters
        turn_left(20); //turn left 1 degree
        set_game_state(1); //transition to game state 1 - DRIVE FORWARD
    }
    else if (game_state==4){ //COLOR CHANGE DETECTED
        drive_backward(.03); // drive backwards 5 centimeters
        if(get_color()==2){
            turn_left(80); //turn around
        }
        else{
            turn_left(160); //turn around
        }
        drive_forward(.01); //drive forward
        set_game_state(1); //transition to game state 1 - DRIVE FORWARD
    }
}

// Pin Change Interrupt for PCINT4 (Pin 12 )
//Corresponds to PCINT0

//Blue Filter 100% Scaling
// S0 High S1 High
// S2 Low S3 High
int previous_color = 0; //used to determining color change
float time_per_tic = 1.0 / 16000000.0; // 1/16 MHZ
float output;
bool hasTurned = 0;
void color_sensor_monitor_setup(){
    TCCR1A = 0b00000000; //Normal Mode Operation

```

```

    TCCR1B = 0b00000001; // No Prescale on Timer 1
    TCNT1 = 0; //Resets TIMER1 to Zero
    PCMSK0 = 1 << 4; // Enable 4 and PCINT1 (PB4)
    PCICR = 0b00000001; //Pin Change Interrupt Control Register -Enable
PCINT0 ISR
    sei(); // enable interrupts
    previous_color = get_color();
}

void color_sensor_monitor_execute(){

    if (previous_color != get_color() && get_game_state() != 4){ //if there
has been a color change and the robot is not currently in game state COLOR
CHANGE transition to color change
        set_game_state(4);
        Serial.println("turn");
        hasTurned = 1;
    }
    previous_color = get_color(); //update previous color after performing
logic
}

void colorInterrupt(){ //GETS CALLED BY Interrupt Service routine for
PORTB pins ISR(PCINT0_vec)
    if(PINB & 1 << 4){ //If PB4 is High
        TCNT1 = 0; //reset timer on falling edge
    }
    else{
        output = 1/(TCNT1*time_per_tic); //measure duration of the pulse using
timer 1
    }
}

int get_color(){ //returns the color the sensor is detecting
    Serial.println(output);
    if(output > 15000){ //pulse duration that corresponds to the color yellow
        return 2;
    }
    return 1; //otherwise color sensor is detecting blue or black
}

```

```

int dt = 100; // delay time for loop
int maxpwm = 18; // maximum value for OCR2B register
int minpwm = 6; // minimum value for OCR2B register

//pg 205
//PWM Phase Correct OCR2A top (Mode 1)
// WGM22 1
// WGM21 0
// WGM20 1

//REGISTERS
//TCCR2A pg 203
//Bit:   7       6       5       4       3       2       1       0
//      COM2A1  COM2A0  COM2B1  COM2B0  /////  ///// WGM21  WGM20

//TCCR2B pg 203
//Bit:   7       6       5       4       3       2       1       0
//      /////  /////  /////  /////  WGM22          [CS2[2:0]]

//From Table 22-5 pg 204

//COM2A1: 1 COM2A0: 0
//Clear OC2A on Compare Match when up-counting. Set OC2A on Compare Match
when down-counting

void door_control_setup(){
  DDRD |= 0b00001000; // set pin 3 as output
  DDRB |= 0b00001000; // set pin 11 as output
  // datasheet p203
  // set timer to mode 1
  // set prescaler to 1024
  // set OC2B to clear on up-counting
  TCCR2A = 0b00100001;
  TCCR2B = 0b00001111;
  // set the OCR2A register. This is the TOP value for timer mode 5
  // (the value to which the timer will count up and then back down)

```

```

    // this is selected to give a period of 20 milliseconds using the
equation
    // period = clock tick * 2 * TOP * prescaler
    // where: one clock tick is 62.5ns and the factor of 2 is because the
timer counts up then back down in PWM mode
    // you only need to set this once at the beginning
    OCR2A = 156;
    open();
    variable_delay(0.1);
    close();
    // now, use the OCR2B register to move the servo

    // the max and min values are selected to oscillate between pulses of
roughly 0.6 and 2.4ms
    // note that OCR2B must be an integer, so this gives you somewhat poor
angular resolution with an 8-bit timer
    // very roughly, you can only move the servo in ~10 degree increments
(about 18 steps over a 180 degree range)
}

void open(){
    OCR2B = minpwm; //Sets OCR2B to pwm signal that corresponds to the
open position
}

void close(){ //Sets OCR2B to pwm signal that corresponds to the closed
position
    OCR2B = maxpwm;
}

bool leftEdge = false; //whether or not a edge has been detected on to
left of the robot
bool rightEdge = false; //whether or not a edge has been detected on to
right of the robot
int lastPinState = 0; //Saves the state of all pins on PINB which includes
pins 8 and 9, the output pins of the QTI sensors

void edge_sensor_setup(){
    DDRB &= 0b11111100; // pins 8 & 9 as inputs using masking

    // Enable Pin Change Interrupts for PCINT0-7 (PORTB)

```

```

PCICR |= 0b00000001;

// Enable PCINT0 and PCINT1 (pins 8 and 9)
PCMSK0 |= 0b00000011;;
sei(); //enables interrupts globally
}

void edge_sensor_execute(){

    if(leftEdge){ //if the left edge was detected change game state to
avoid left edge
        leftEdge = false;
        set_game_state(2);
    }

    if(rightEdge){ //if the right edge was detected change game state to
avoid right edge
        rightEdge = false;
        set_game_state(3);
    }
}

bool getLeftEdge(){ //getting function for leftEdge
    return leftEdge;
}

bool getRightEdge(){ //getting function for rightEdge
    return rightEdge;
}

void edgeInterrupt(){ //GETS CALLED BY Interrupt Service routine for PORTB
pins ISR(PCINT0_vec)

    uint8_t currentState = PINB; //updates current state by polling PINB

    // PIN 8 = PB0
    if ((currentState & 0b00000001) && !(lastPinState & 0b00000001)) {
//if pin 8 is high and it was not before
        // rising edge on pin 8
        leftEdge = true; //update leftEdge
    }
}

```

```
        Serial.print("leftEdge");
    }

    // PIN 9 = PB1
    if ((currentState & 0b00000010) && !(lastPinState & 0b00000010)) {
//if pin 9 is high and it was not before
        // rising edge on pin 9
        rightEdge = true; //update rightEdge
        Serial.print("rightEdge");
    }

    lastPinState = currentState; //updates last pin state only after
having executed the logic for updating leftEdge and rightEdge
}
```